NetOps Manager

1. Bài toán thực tế

Giả sử có rất nhiều server:

- AWS EC2
- Azure
- GCP
- VPS
- OpenStack VM (UITIOT Cloud)
- Home Lab (máy ảo chạy trên PC)

Bình thường muốn quản lý server thì Laptop phải SSH vô từng máy rồi dùng các lệnh như: top, free -h, df -h, docker ps (để xem có container và trạng thái của chúng ra sao), systemctl status nginx (trạng thái của service nginx đang chạy trên server).

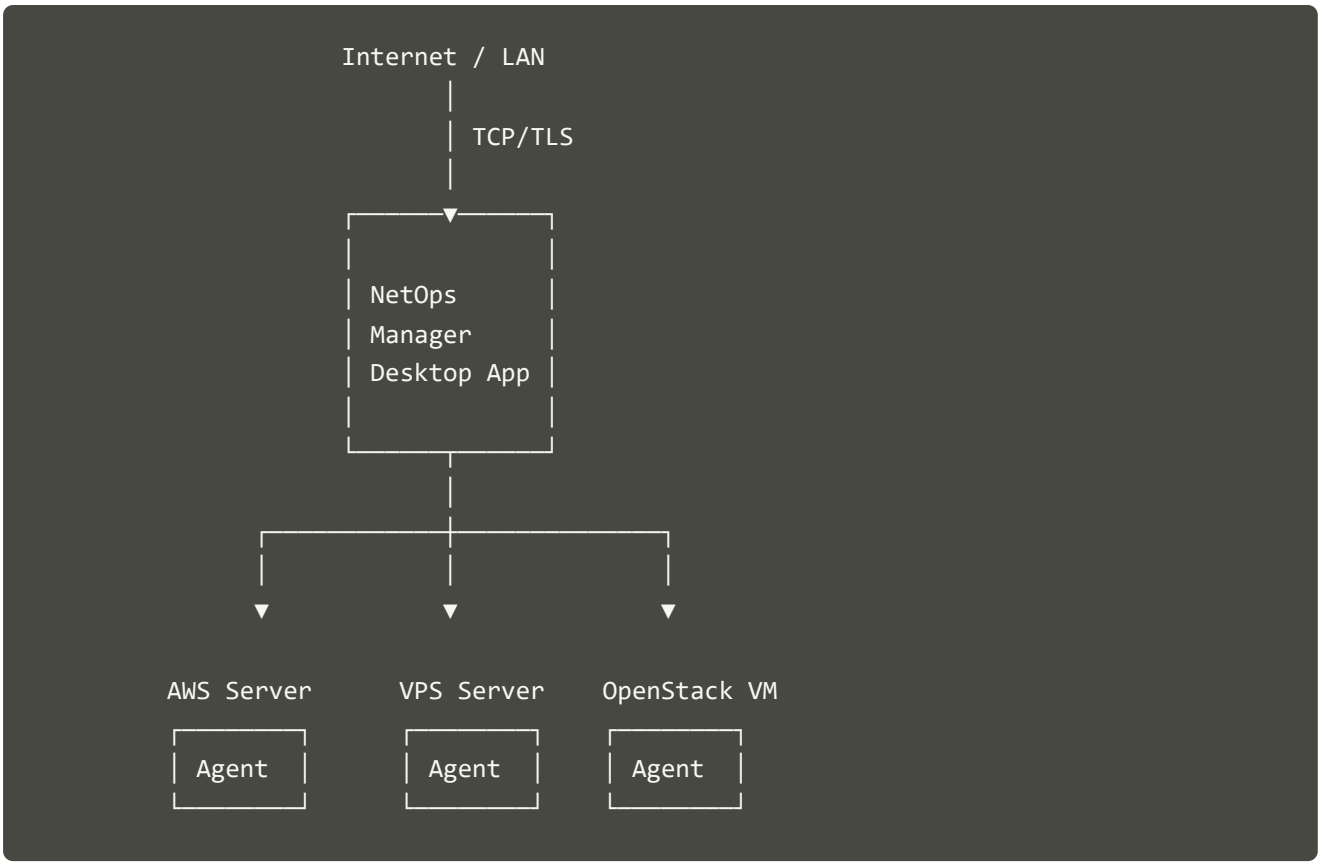
Thực tế thì không doanh nghiệp nào quản lý 1 server cả, họ dùng nhiều server để triển khai sản phẩm với mục tiêu là đảm bảo HA (tính sẵn sàng cao), Auto Scaling → Mục đích chính của Auto scaling là tối ưu chi phí cho doanh nghiệp với các máy ảo chạy từ các CSP lớn (pay as you go). Vậy nên ứng dụng trên sinh ra với mục đích là quản lý nhiều máy ảo hoạt động trên một nơi duy nhất. Vậy ứng dụng này NÊN quản lý cái gì?

- Server status
- CPU, RAM
- Dung lượng Disk
- Docker container status nằm trong server đó
- Services status
- Port (chỉ định) nào server không không vào được
- Alert khi có server vừa disconnect

Mấy cái metric trên thông thường thì chạy lệnh để lấy nhưng bây giờ phải làm cách nào đó để nó tự động chạy trong mỗi 1p hoặc 5p để nó chạy mấy cái lệnh này và cũng bằng cách nào đó gửi nó về server quản lý. Server quản lý sẽ lưu mấy cái metric này (trong database), mục tiêu là có thể dùng để phân tích (chưa biết là phân tích cái gì, có thể bỏ database cũng được luôn). Sau đó biến mấy cái metric thành API.

Dùng C# viết fontend với backend cho DashBoard, rồi kéo API từ server về để nó hiện metric lên cái app.

Mỗi server sẽ cài một chương trình nhỏ gọi là:
NetOps Agent
Kiến trúc:



Tính năng khác

Thay vì SSH bằng lệnh thì giờ làm tính năng trên ứng dụng để tự gọi SSH truy cập đơn giản hơn (Giống với Teleport Service → Một ứng dụng dùng trong doanh nghiệp vì SSH thông thường vừa thiếu bảo mật, vừa thiếu chuyên nghiệp)

Hoặc thay vì SSH thì hiện terminal của server lên app cũng được

---

11. Streaming Logs

Đây cũng là tính năng networking rất hợp.

Ví dụ:

```
backend container
```

đang log:

```
14:20 GET /api/login 200
14:21 GET /api/users 200
14:21 DB connection failed
14:22 retrying...
```

Manager có thể subscribe:

```
SUBSCRIBE_LOG
container=backend
```

Agent:

```
tail -f log
```

và liên tục push:

```
Agent
 │
 ├── log line
 ├── log line
 ├── log line
 ↓
Manager
```

Người dùng nhìn realtime.

---

12. File Manager

Tab:

```
Files
```

có thể giống WinSCP:

```
Server: AWS-Web-01

/root/project/

├── compose.yaml
├── Dockerfile
├── .env
├── app/
└── logs/
```

Người dùng có thể:

```
Upload

Download

Delete

Rename
```

Nhưng quan trọng là nhóm có thể **tự viết file transfer protocol**.

Ví dụ file:

```
ubuntu.iso
500 MB
```

không gửi:

```
500 MB một cục
```

mà chia:

```
Chunk 1
Chunk 2
Chunk 3
...
Chunk 500
```

---

13. Nếu mạng rớt khi upload thì sao?

Đây là một use case cực kỳ hay cho NT106.

Ví dụ:

```
500 MB
```

đã gửi:

```
0 → 327 MB
```

rồi:

```
❌ mất mạng
```

App không bắt gửi lại từ 0.

Khi reconnect:

```
Agent:

"Tôi đã nhận đến chunk 327."
```

Manager:

```
OK.

Resume từ chunk 328.
```

Tiếp tục:

```
328
329
330
...
500
```

Đây là:

```
Resumable File Transfer
```

và rất dễ demo.

---

14. Checksum để làm gì?

Ví dụ bạn gửi:

```
app.zip
```

Manager tính:

```
SHA-256
```

giả sử:

```
ABCD123...
```

Server nhận xong cũng tính SHA-256.

Nếu:

```
Manager:
ABCD123

Agent:
ABCD123
```

→

```
✅ File chính xác
```

Nếu:

```
Manager:
ABCD123

Agent:
892XYZ
```

→

```
❌ File corrupted
```

đây chính là kiểm tra **integrity**.

17. Quản lý nhiều server cùng lúc

Đây là điểm app bắt đầu có lợi hơn SSH thông thường.

Ví dụ:

```
10 web servers
```

bạn muốn:

```
docker pull myapp:v3
```

SSH truyền thống:

```
Server 1 → chạy
Server 2 → chạy
Server 3 → chạy
...
```

NetOps:

```
Select:

☑ web-01
☑ web-02
☑ web-03
☑ web-04

Command:

docker pull myapp:v3

[Run]
```

Manager gửi song song:

```
          ┌── Agent 01
          ├── Agent 02
Manager ──┼── Agent 03
          └── Agent 04
```

kết quả:

```
web-01 ✅

web-02 ✅

web-03 ❌

web-04 ✅
```

Rất hợp với DevOps.

---

18. Đây là lúc concurrency xuất hiện

Manager có thể đang giữ:

```
20 connections
```

đồng thời.

```
Server 01 ─┐
Server 02 ─┤
Server 03 ─┤
Server 04 ─┤
...        ├── Manager
Server 20 ─┘
```

Không thể xử lý kiểu:

```
xong server 1
↓
mới server 2
↓
mới server 3
```

mà cần:

```
async

thread

task

event-driven
```

tùy công nghệ nhóm dùng.

Đây cũng chính là kiến thức lập trình mạng.

---

19. Reconnect là một feature quan trọng

Ví dụ:

```
AWS Server
     |
     |
     X
 Internet mất
```

Agent phát hiện:

```
connection closed
```

Không crash.

Nó chuyển sang:

```
Reconnect...

2 seconds

4 seconds

8 seconds
```

Sau khi Internet có lại:

```
Agent ──────── Manager
```

Agent authenticate lại:

```
server_id

token
```

sau đó tiếp tục heartbeat.

---

20. Authentication

Không thể để:

```
bất kỳ Agent nào
```

kết nối Manager.

Ví dụ khi thêm server:

```
Manager tạo:

Agent Token
```

```
srv_9fd81a...
```

Bạn cài trên server:

```
netops-agent --token=srv_9fd81a...
```

Agent kết nối:

```
AUTH

server_id = srv01

token = srv_9fd81a
```

Manager kiểm tra.

Đúng:

```
AUTH_OK
```

Sai:

```
AUTH_FAILED
```

---

21. TLS để bảo vệ kết nối

Nếu Manager gửi:

```
docker restart backend
```

mà gửi plaintext thì người nghe lén mạng có thể đọc được.

Vì vậy:

```
Agent
   |
   | TLS encrypted connection
   |
Manager
```

Từ đó có thể nói trong báo cáo:

```
Confidentiality

Integrity

Authentication
```

---

22. Một ngày sử dụng app thực tế sẽ như thế nào?

Giả sử bạn deploy website lên AWS.

Sáng mở NetOps:

```
AWS-Web
🟢 Online

CPU 17%

RAM 44%

Disk 62%
```

Bạn mở:

```
Docker
```

thấy:

```
backend
🔴 Exited
```

Bạn click:

```
Logs
```

thấy:

```
Database connection refused
```

Bạn kiểm tra:

```
Services
```

thấy:

```
PostgreSQL
🔴 Failed
```

Click:

```
Restart PostgreSQL
```

sau đó:

```
PostgreSQL
🟢 Running
```

Restart backend:

```
backend
🟢 Running
```

Website chạy lại.

Toàn bộ không cần:

```
ssh

systemctl

docker ps

docker logs

docker restart
```

thủ công.

Đó chính là giá trị thực tế của app.

---

23. Một demo đồ án có thể cực kỳ dễ hiểu

Khi trình bày trước giảng viên, bạn có thể chuẩn bị:

```
Laptop
   │
   │
   ├── VM Ubuntu 1
   ├── VM Ubuntu 2
   └── AWS EC2
```

Trên dashboard:

```
3 Servers Online
```

Sau đó tắt một VM:

```
sudo shutdown now
```

Dashboard:

```
VM 2

🟢 Online
    ↓
🟡 Connection Lost
    ↓
🔴 Offline
```

Sau đó bật lại:

```
Agent reconnect

🔴 Offline
    ↓
🟢 Online
```

Tiếp theo:

```
Terminal
```

chạy:

```
uname -a
```

output realtime.

Tiếp:

```
upload file 200 MB
```

giữa chừng:

```
ngắt mạng
```

sau đó reconnect và:

```
resume upload
```

Cuối cùng:

```
restart Docker container
```

Một demo như vậy sẽ thể hiện rất rõ:

```
Socket

TCP

Client-server

Concurrency

Heartbeat

Reconnect

File transfer

Streaming

Authentication
```

chứ không chỉ là giao diện CRUD.

---

24. Những tính năng nên coi là CORE

Nếu nhóm không muốn scope quá lớn, mình nghĩ phiên bản đầu tiên nên tập trung vào **8 tính năng cốt lõi**:

1. Thêm và quản lý nhiều server.
2. Agent kết nối Manager bằng TCP.
3. Heartbeat + online/offline detection.
4. Realtime CPU/RAM/Disk/network metrics.
5. Remote command execution.
6. Streaming command output/log.
7. File upload/download có progress.
8. Authentication + TLS.

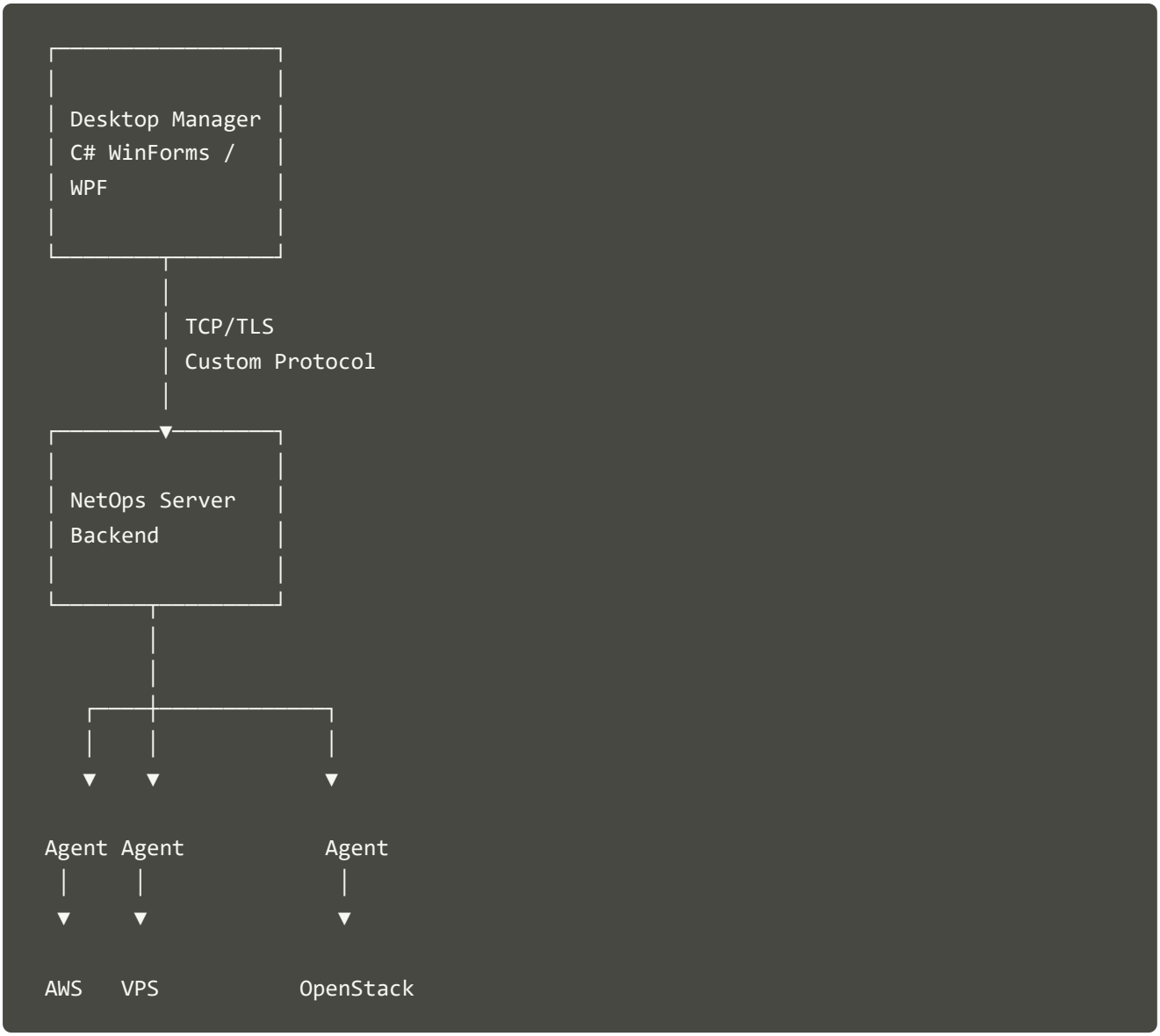
Nếu làm được 8 cái này **chắc chắn và đẹp**, đồ án đã ổn.

Docker Manager, port monitor, resume file transfer, alert, multi-server command... có thể trở thành phần mở rộng.

---

25. Phân chia app thành 3 thành phần sẽ rất dễ hiểu

Mình đề xuất kiến trúc cuối cùng:



Hoặc nếu muốn đơn giản hơn:



Mình **nghiêng về kiến trúc thứ hai cho NT106** vì nhóm sẽ trực tiếp xử lý socket giữa Manager và Agent, dễ chứng minh kiến thức mạng hơn.

---

26. Điểm khác biệt giữa nó và các phần mềm SSH

Nếu giảng viên hỏi:

> “Sao không dùng Termius?”

Bạn có thể trả lời theo hướng:

Termius chủ yếu cung cấp SSH client.

NetOps Manager tập trung vào:

```
Continuous server connection

Realtime monitoring

Heartbeat

Multiple server management

Custom agent protocol

Streaming

Remote service management

Resumable transfer

Centralized status
```

Nó giống một hệ thống:

```
Remote Administration
+
Monitoring
+
Network Management
+
DevOps Tools
```

hơn là chỉ SSH client.

---

27. Và quan trọng nhất: nó rất hợp định hướng của bạn

Bạn đang đi hướng:

```
Cloud

DevOps

Server

AWS

Docker

CI/CD
```

thì đồ án này không phải kiểu:

> Làm xong môn rồi bỏ.

Bạn hoàn toàn có thể tiếp tục phát triển nó thành tool cá nhân.

Sau này có thể thêm:

```
AWS EC2 discovery

Docker Compose deployment

GitHub Actions

Deployment history

Rollback

Alerts

Telegram/Discord notification

Server groups

Prometheus integration

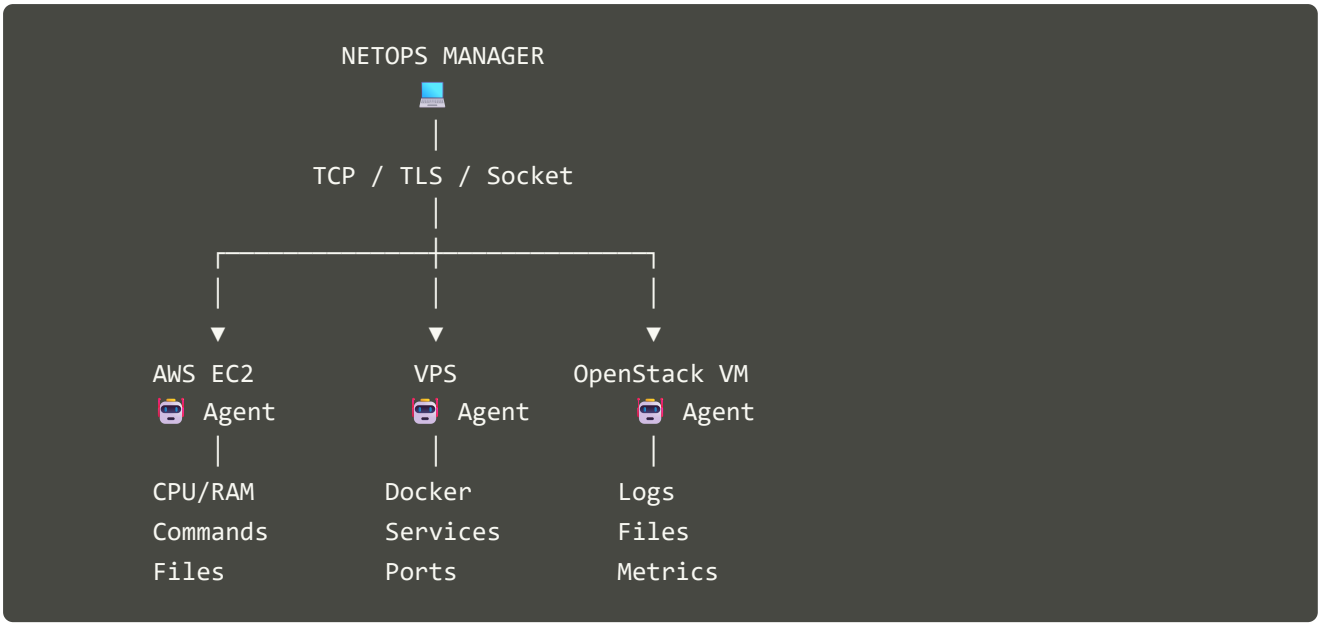
Kubernetes monitoring
```

Từ một đồ án NT106, nó có thể dần biến thành một **mini DevOps control center**.

Nếu tóm lại app trong đúng một câu

> **NetOps Manager là ứng dụng desktop cho phép người dùng giám sát và điều khiển nhiều Linux server từ xa theo thời gian thực thông qua hệ thống Agent và giao thức mạng client–server, hỗ trợ theo dõi trạng thái, thực thi lệnh, truyền file, xem log và quản lý dịch vụ/server tập trung.**

Và nếu phải tóm nó bằng một hình duy nhất thì chính là:



Đây mới là hình dung mình muốn nhóm bạn giữ trong đầu: không phải "app SSH có giao diện đẹp", mà là **một hệ thống điều khiển và giám sát server phân tán qua mạng**.